Exercise Sheet 5

Community Detection

5942UE Network Science

Prof. Dr. Michael Granitzer Dr. Kanishka Ghosh Dastidar Dr. Jelena Mitrovic

5.A Basic Measures and Modularity

Exercise 5.A.1: Community Density by Hand

Graph: A, B, C, D, E, F, edges A–B, A–C, B–C, B–D, D–E, D–F, E–F.

Two candidate partitions:

- **P1:** A, B, C, D and E, F
- **P2:** A, B, C and D, E, F

For each partition:

1. Compute **internal edge density**: $\left|\text{edges inside}\right|$ / $\left|\text{possible edges inside}\right|$.
2. Compute **external edge density**: $\left|\text{crossing edges}\right|$ / $\left|\text{possible crossing edges}\right|$.
3. Which partition better separates the network? Justify with numbers.
4. What is the "ground truth" community structure visible from inspection?

Solution:

Partition P1: A, B, C, D and E, F

- Internal edges in A, B, C, D: A–B, A–C, B–C, B–D = 4. Possible: $\binom{4}{2} = 6$. Density = $4/6 \approx 0.67$.
- Internal edges in E, F: E–F = 1. Possible: $\binom{2}{2} = 1$. Density = $1/1 = 1.0$.
- Crossing edges: D–E, D–F = 2. Possible crossing: $4 \times 2 = 8$. External density = $2/8 = 0.25$.

Partition P2: A, B, C and D, E, F

- Internal edges in A, B, C: A–B, A–C, B–C = 3. Possible: $\binom{3}{2} = 3$. Density = $3/3 = 1.0$.
- Internal edges in D, E, F: D–E, D–F, E–F = 3. Possible: $\binom{3}{2} = 3$. Density = $3/3 = 1.0$.
- Crossing edges: B–D = 1. Possible: $3 \times 3 = 9$. External density = $1/9 \approx 0.11$.

Comparison: P2 achieves perfect internal density (1.0) in both communities and minimal external density (0.11). P1 mixes D into the left community even though D is more tightly connected to E and F than to A and C. P2 is the clearly superior partition.

Ground truth: The graph consists of two complete triangles (A–B–C and D–E–F) joined by a single bridge edge B–D. The natural communities are exactly A, B, C and D, E, F.

Exercise 5.A.2: Modularity — Concept and Computation

Modularity Q measures whether communities are denser than expected by chance:

$$Q = \sum_c \left(\frac{l_c}{m} - \left(\frac{d_c}{2m} \right)^2 \right)$$

where l_c = edges inside community c, d_c = sum of degrees in community c, m = total edges.

1. For **P2** from Exercise 5.A.1, compute Q by hand (use $m = 7$).
2. What does $Q = 0$ mean? What does $Q = 1$ mean? What range is typical for real networks?
3. If you randomly shuffled community assignments, what would you expect Q to approach?

Solution:

1. Computing Q for P2 ($m = 7$ total edges):

Community A, B, C: internal edges $l_1 = 3$. Degrees: $A = 2, B = 3, C = 2 \to d_1 = 7$.

Community D, E, F: internal edges $l_2 = 3$. Degrees: $D = 3, E = 2, F = 2 \to d_2 = 7$.

$$Q = \left(\frac{3}{7} - \left(\frac{7}{14}\right)^2\right) + \left(\frac{3}{7} - \left(\frac{7}{14}\right)^2\right)$$

$$Q = 2 \cdot \left(\frac{3}{7} - \frac{1}{4}\right) = \frac{5}{14} \approx 0.357$$

This is a small graph, so Q is positive but not extremely high despite the clear structure. Larger real-world networks with similar community clarity often yield higher Q.

2. Interpreting Q:

- $Q = 0$: community structure is no better than random — no detectable communities.
- $Q = 1$: perfect modularity (theoretically unachievable in practice with real degree sequences).
- Typical real networks: $Q \in [0.3, 0.7]$ for networks with clear community structure.

3. Random assignment: If assignments are random (nodes shuffled into communities), the expected Q approaches 0. Any positive Q indicates more internal edges than chance alone explains.

Exercise 5.A.3: Community Detection in Python

Using `nx.karate_club_graph()`:

1. Apply greedy modularity maximisation: `nx.community.greedy_modularity_communities(G)`.
2. Compute the modularity score of the result using `nx.community.modularity(G, communities)`.
3. Compare to the known faction labels (`G.nodes[n]["club"]`). What fraction of nodes are assigned to the "correct" faction?
4. Visualise: plot the network with nodes coloured by detected community.

Solution:

```
import networkx as nx
import matplotlib.pyplot as plt

G = nx.karate_club_graph()
communities = nx.community.greedy_modularity_communities(G)
Q = nx.community.modularity(G, communities)

print(f"Modularity Q = {Q:.3f}, Communities = {len(communities)}")

def majority_vote_accuracy(G, node_to_group):
    correct = 0
    for group in set(node_to_group.values()):
        members = [n for n, g in node_to_group.items() if g == group]
        labels = [G.nodes[n]["club"] for n in members]
        majority = max(set(labels), key=labels.count)
        correct += sum(label == majority for label in labels)
    return correct / G.number_of_nodes()
```

Interpretation:

5.B Girvan-Newman Algorithm

Exercise 5.B.1: Girvan-Newman Steps

Graph: 6 nodes, two triangles $1, 2, 3$ and $4, 5, 6$ connected by single edge 3–4.

1. Compute **edge betweenness** for all edges. Which edge has the highest value?
2. Remove that edge. What happens to the graph?
3. Which edges now have the highest betweenness after the removal?
4. Why is edge 3–4 guaranteed to have the highest betweenness in this structure?

Solution:

1. Edge betweenness:

Edge 3–4 lies on every path between a node in $1, 2, 3$ and a node in $4, 5, 6$: $3 \times 3 = 9$ pairs. No other single edge is on as many shortest paths. All edges within the triangles mediate at most 2–4 paths. Edge 3–4 has betweenness = 9 (un-normalised), far above any intra-triangle edge.

2. After removing 3–4:

The graph splits into two disconnected triangles: $1, 2, 3$ and $4, 5, 6$. All inter-cluster paths no longer exist.

3. Highest betweenness after removal:

Within each isolated triangle, all three edges are equivalent. Each edge lies on exactly one shortest path: the direct path between its two endpoints. So every intra-triangle edge has the same unnormalised betweenness, equal to 1.

4. Why 3–4 is guaranteed highest:

Any path from any node in $1, 2, 3$ to any node in $4, 5, 6$ must cross this edge — there is no alternative route. It is the unique cut edge (bridge) between the two components. The betweenness of a bridge is always $|L| \times |R|$ where $|L|$ and $|R|$ are the sizes of the two components it separates, giving $3 \times 3 = 9$ here.

Exercise 5.B.2: Girvan-Newman in Practice

Using `nx.karate_club_graph()`:

1. Apply the Girvan-Newman algorithm: `nx.community.girvan_newman(G)`.
2. Extract the partition at the 2-community level. Does it recover the known factions?
3. Compare faction recovery accuracy to the greedy modularity result from Exercise 5.A.3.
4. Plot the edge betweenness of the top-5 edges at the initial state and after 1 removal step.

Solution:

```
import networkx as nx
import matplotlib.pyplot as plt

G = nx.karate_club_graph()
comp = nx.community.girvan_newman(G)
top2 = list(next(comp))  # 2-community level

def majority_vote_accuracy(G, node_to_group):
    correct = 0
    for group in set(node_to_group.values()):
        members = [n for n, g in node_to_group.items() if g == group]
        labels = [G.nodes[n]["club"] for n in members]
        majority = max(set(labels), key=labels.count)
        correct += sum(label == majority for label in labels)
    return correct / G.number_of_nodes()

gn_assignment = {}
for i, community in enumerate(top2):
```

Interpretation:

- **Precision:** Girvan-Newman at 2 communities usually matches the known faction split very closely. Unlike modularity, it is forced here to return exactly two groups.
- **Top-down logic:** By iteratively removing the highest-betweenness (bridge) edges, the algorithm cleanly isolates the two social hubs (Mr. Hi and the Officer) and their respective followers.

5.C Hierarchical Clustering

Exercise 5.C.1: Dendrogram Interpretation

Consider the 6-node two-triangle graph from the Girvan-Newman exercise.

1. What does the **height** of a merge step in a dendrogram represent?
2. In the dendrogram for this graph, at what level do the two triangles merge into a single cluster?
3. How would you choose the "right" number of clusters from a dendrogram?
4. What is the difference between using **direct-link similarity** vs. **neighbourhood-overlap similarity** as the merge criterion?

Solution:

1. Height of a merge: The height represents the dissimilarity (or distance) between the two clusters being merged at that step. A merge at low height means two very similar clusters were joined; a merge at high height indicates two dissimilar clusters being forced together.

2. Merge level for two-triangle graph: Within each triangle, nodes share neighbours and direct links, so they merge at low height first. The two triangles merge at the highest height because they are connected only by a single bridge edge — maximum dissimilarity.

3. Choosing the number of clusters: Look for the largest **gap** in merge heights (the longest vertical line in the dendrogram before the next merge). Cutting just below a large gap gives a natural number of clusters. Here, the large gap is just below the inter-triangle merge, suggesting $k = 2$.

4. Direct-link vs. neighbourhood-overlap:

- *Direct-link similarity*: two nodes are similar if they share an edge. Merges nearby nodes but ignores global structure.
- *Neighbourhood-overlap*: similarity = $|N(u) \cap N(v)|/|N(u) \cup N(v)|$. Two nodes with many shared neighbours are similar even without a direct link. This is more robust to missing edges and better captures social equivalence.

Exercise 5.C.2: Hierarchical Clustering in Python

Using `nx.karate_club_graph()` and `scipy`:

1. Compute pairwise **shortest-path distances** between all nodes. Use this as the distance matrix.
2. Apply hierarchical clustering with average linkage: `scipy.cluster.hierarchy.linkage(...)`.
3. Plot the resulting dendrogram.
4. Cut the dendrogram to obtain 2 clusters. What fraction of nodes match the known factions?

Solution:

```
import networkx as nx
import numpy as np
from scipy.cluster.hierarchy import linkage, fcluster, dendrogram
from scipy.spatial.distance import squareform
import matplotlib.pyplot as plt

G = nx.karate_club_graph()
nodes = list(G.nodes())
n = len(nodes)

# 1. Distance matrix from shortest paths
D = np.zeros((n, n))
sp = dict(nx.all_pairs_shortest_path_length(G))
for i, u in enumerate(nodes):
    for j, v in enumerate(nodes):
        D[i, j] = sp[u][v]

# 2. Average linkage
```

Interpretation:

- **Dendrogram structure:** The dendrogram shows two main branches merging at the highest height — these correspond to the two factions.
- **Robustness:** Average linkage with shortest-path distances captures broad social proximity, but some fringe bridge-nodes may be misclassified if they sit close to both centers in terms of hops.